Multilingual Citizen Grievance Chatbot

Complete Execution Plan and Technical Scope

This document defines the complete development plan for HI-02: a multilingual citizen grievance chatbot that accepts complaints in Indian languages, identifies the correct department, creates and routes tickets, tracks resolution status, and notifies citizens. It incorporates the requirements in the supplied project brief and hackathon guide, including Hugging Face models, LangChain orchestration, Whisper voice input, FastAPI, PostgreSQL, Telegram, web fallback, admin operations, escalation, evaluation and data preparation.

The implementation will be delivered in working stages. The text-based MVP is the first release target; voice, analytics and additional integrations will be added only after the complete ticketing and status workflow is stable.

1 Project objective

Build an inclusive grievance intake and tracking system for citizens who prefer regional Indian languages. The system must accept text and voice complaints, understand code-mixed language, classify each complaint, route it to the appropriate department, create a traceable ticket, and keep the citizen informed until resolution.

2 Requirements covered

Requirement	Planned implementation	Priority
Hugging Face	Use multilingual transformer models and embeddings for language-aware classification and translation.	Core
LangChain	Use for conversation orchestration, clarification prompts and optional LLM provider integration. Core ticket logic remains deterministic.	Core integration
Whisper voice	Transcribe Telegram voice notes using Whisper or faster-whisper, then send the transcript through the same NLP pipeline.	Phase 8
FastAPI	Backend APIs, web chat endpoint, ticket service, admin endpoints and health checks.	Core
PostgreSQL	Primary production database for users, tickets, departments, events, notifications and audit records.	Core
Telegram	Primary citizen-facing chatbot with language selection, complaint intake, status lookup and notifications.	Core
Data ideas	Synthetic multilingual complaints, AI4Bharat resources, public grievance categories and handwritten evaluation data.	Core

3 Supported languages and complaint types

- English, Hindi and Telugu as the initial supported languages.
- Hinglish and Tenglish code-mixed messages written in Roman script.
- Automatic script and language detection with user confirmation when confidence is low.
- Text complaints in the MVP and voice complaints through Whisper in the voice phase.
- Optional location or landmark capture to help departments act on the complaint.

4 End to end system flow

Citizen message -> language and script detection -> voice transcription when required -> text normalization -> Hugging Face multilingual classification -> LangChain conversation decision -> clarification if confidence is low -> translation or English summary for officers -> ticket creation -> department routing -> officer status update -> citizen notification -> SLA monitoring and escalation.

Classification will use the original multilingual complaint whenever possible. Translation will be used to provide an officer-facing summary and localized responses, preventing translation errors from becoming the only source of classification information.

5 Technology architecture

Layer	Technology	Purpose
Citizen interfaces	Telegram Bot API and responsive web chat	Complaint submission, language selection, ticket lookup and notifications.
API backend	FastAPI and Pydantic	REST endpoints, validation, authentication boundaries and service integration.
Conversation layer	LangChain with deterministic Python state machine	Conversation memory, clarification flow and optional LLM-assisted prompts.
Language and ML	Hugging Face Transformers, sentence-transformers, MuRIL/IndicBERT or multilingual-e5	Language-aware embeddings, department prediction, priority prediction and confidence scoring.
Translation	IndicTrans2 adapter with configured fallback	Officer summaries and citizen responses in supported languages.
Voice	Whisper or faster-whisper	Speech-to-text for Telegram voice messages.
Database	PostgreSQL with SQLAlchemy	Persistent tickets, users, departments, events, notifications and audit history.
Local fallback	SQLite	Simple offline setup and reliable demonstration mode.
Operations	APScheduler initially; separate worker option for production	SLA checks, escalation and notification delivery.
Deployment	Docker Compose initially	Repeatable local and demo deployment.

6 Functional modules

6.1 Language detection

- Detect Devanagari Hindi, Telugu script and Roman-script English/Hinglish/Tenglish.
- Mark code-mixed messages and allow manual language correction.
- Handle empty, emoji-only, gibberish and unsupported-language inputs gracefully.

6.2 Voice intake

- Receive Telegram voice notes.
- Transcribe them with Whisper or faster-whisper.
- Show the transcript for confirmation before filing when confidence is low.
- Preserve the transcript and original message metadata according to privacy settings.

6.3 Complaint classification

- Predict department, intent and priority.
- Return confidence and model version.
- Use keyword rules for urgent cases and model failures.
- Ask a department clarification question below the confidence threshold.

6.4 Translation and summaries

- Use IndicTrans2 through a lazy-loaded adapter.
- Cache translations.
- Keep original complaint text unchanged.
- Display an officer-facing English summary and localized citizen responses.

6.5 Ticket management

- Generate unique IDs such as GRV-2026-000123.
- Create, assign, update, resolve, close and reopen tickets.
- Record every status change as an event.
- Support department and priority filtering.

6.6 Citizen experience

- Language selection through /start.
- Complaint submission and confirmation.
- Ticket ID response.
- Status lookup and recent ticket history.
- Localized notifications and help/cancel commands.

6.7 Admin operations

- Secure officer login.
- Department queue, filters and search.
- Ticket detail page with original text, translation, confidence and event history.
- Status update, officer note and escalation visibility.

6.8 SLA and escalation

- Department-specific SLA hours.
- High-priority SLA reduction.
- Automatic escalation for overdue unresolved tickets.
- Escalation events and citizen notification.
- Demo mode with one- or two-minute SLAs.

7 Department categories

Code	Department	Example complaints
WATER	Water Supply and Sewerage	Leaks, no water, drainage and sewage overflow.
POWER	Electricity	Power outage, damaged poles, exposed wires and billing issues.
ROADS	Roads and Infrastructure	Potholes, broken roads, traffic signals and streetlights.
SANITATION	Sanitation and Solid Waste	Garbage collection, dumping and public cleanliness.
POLICE	Police and Public Safety	Harassment, threats, accidents and urgent safety issues.
HEALTH	Health	Public health facilities, sanitation risks and emergency concerns.
EDUCATION	Education	School services, certificates and public education facilities.
TRANSPORT	Transport	Bus services, permits, road transport and public transit.
REVENUE	Revenue, Land and Certificates	Land records, certificates and civic documentation.
OTHER	Other or Human Review	Low-confidence, cross-department or unsupported complaints.

8 Data plan

The initial model will use a structured multilingual complaint corpus. Synthetic data is appropriate for the hackathon prototype, but results will be reported separately for generated data and realistic handwritten test messages. Public resources will be used only after checking their license and attribution requirements.

Data source	Planned use	Controls
Synthetic complaint corpus	60-100 examples per department across English, Hindi, Telugu, Hinglish and Tenglish.	Deduplicate, balance classes and hold out a test split.
Handwritten evaluation set	30-40 realistic, messy complaints for demo and robustness testing.	Keep separate from training data.
AI4Bharat datasets	Translation and multilingual NLU references such as Samanantar, IN22 and IndicXNLI.	Check dataset license and cite sources.

Data source	Planned use	Controls
Public grievance categories	Department names, intents and example category structure.	Use only permitted text; do not copy protected personal information.
Hugging Face models	MuRIL, IndicBERT, multilingual-e5 and IndicTrans2 components.	Record exact model names, versions and licenses.

Each record will contain text, language, department, intent and priority. The evaluation script will produce macro-F1, accuracy, per-language results, a confusion matrix and the percentage of messages sent to clarification.

9 Database design

Entity	Important fields
users	id, Telegram ID, name, preferred language, created_at
departments	id, name, SLA hours, escalation contact, active
tickets	id, ticket code, user, original text, language, summary, department, intent, priority, confidence, status, location, SLA deadline, escalation level, timestamps
ticket_events	ticket, old status, new status, note, actor, created_at
notifications	ticket, channel, recipient, language, message, delivery status, sent_at
admin_users	id, username, password hash, role, department, active
audit_logs	actor, action, resource, metadata, timestamp

10 API scope

Method	Endpoint	Purpose
GET	/health	Service health and dependency status.
POST	/chat	Process a web chat message and return the next response.
POST	/tickets	Create a ticket after classification and confirmation.
GET	/tickets/{code}	Return ticket status, department and latest update.
PATCH	/tickets/{code}/status	Update status, add note and create an event.
GET	/admin/tickets	List tickets by department, status, priority or search term.

Method	Endpoint	Purpose
POST	/classify	Debug endpoint for language, department, intent, priority and confidence.
POST	/voice/transcribe	Transcribe an uploaded voice message when voice support is enabled.

11 Delivery phases

Phase	Work	Output
1	Inspect workspace, lock scope, create project guide and environment configuration.	Runnable repository skeleton.
2	FastAPI, SQLAlchemy, PostgreSQL/SQLite configuration, models and health endpoint.	Backend foundation.
3	Ticket CRUD, status events, department queues and validation.	Working ticket API.
4	Synthetic data generator, handwritten test set, cleaning, splitting and documentation.	Training and evaluation data.
5	Hugging Face embeddings, department/priority classifiers, confidence logic and rule fallback.	Working multilingual classifier.
6	Language detection, IndicTrans2 adapter, caching and localized strings.	Language and translation services.
7	Telegram bot, LangChain conversation orchestration, clarification flow and status commands.	End-to-end citizen workflow.
8	Whisper voice intake and transcript confirmation.	Voice complaint support.
9	Admin dashboard, officer actions, notifications and escalation worker.	Operational workflow.
10	Web fallback, testing, security hardening and deployment configuration.	Demo-ready application.
11	README, architecture diagram, screenshots, demo script, results and final clean-clone test.	Submission-ready package.

12 Security and reliability requirements

- Store Telegram tokens, database credentials and API keys only in environment variables.
- Protect admin pages and status update endpoints with authentication and role checks.
- Validate and limit message size, uploaded audio size and request frequency.
- Use parameterized database access through SQLAlchemy.
- Preserve original complaint text while minimizing unnecessary personal information.
- Record officer actions and status changes for accountability.
- Use deterministic fallback behavior when translation, model loading or external APIs fail.
- Keep model artifacts and large files out of Git history; provide reproducible download or training instructions.

13 Testing and acceptance criteria

- [ ] A citizen can submit an English, Hindi, Telugu, Hinglish or Tenglish complaint.
- [ ] The system detects or confirms the language and handles unsupported input politely.
- [ ] The system predicts department, intent, priority and confidence.
- [ ] Low-confidence complaints trigger a clarification question.
- [ ] A unique ticket ID is created and returned in the selected language.
- [ ] The ticket appears in the correct admin department queue.
- [ ] An officer can update the status and add a note.
- [ ] The citizen can query the status using the ticket ID.
- [ ] Every status update creates a ticket event and audit record.
- [ ] Overdue tickets are escalated and notifications are sent.
- [ ] Voice messages are transcribed when Whisper support is enabled.
- [ ] The application runs with SQLite for local demonstration and PostgreSQL for the full deployment.
- [ ] Automated tests pass and the project starts from a clean clone using documented commands.

14 Risks and fallback decisions

Risk	Fallback
IndicTrans2 is too slow or too large	Lazy-load it and use a configured translation provider or a deterministic summary fallback.
Whisper is unavailable on the demo machine	Keep text intake fully functional and show voice as an optional enabled deployment profile.
Telegram access is unavailable	Use the web chat fallback and recorded demo workflow.
Telugu classification is weak	Add Telugu/Tenglish examples, improve rules and route uncertain cases to human review.
PostgreSQL is unavailable locally	Use SQLite with the same service interfaces.
Model files are too large for Git	Ignore artifacts and provide a training/download script.
Time is limited	Prioritize ticket creation, department assignment, status lookup and admin updates before analytics.

15 Final deliverables

- Working FastAPI backend with PostgreSQL and SQLite configuration.
- Telegram chatbot with multilingual text, status tracking and localized notifications.
- Whisper-based voice complaint support.
- Hugging Face classification and IndicTrans2 translation services.
- LangChain conversation and clarification orchestration.
- Admin dashboard with department queues and status updates.
- SLA monitoring and automatic escalation workflow.
- Responsive web fallback interface.

- Synthetic and handwritten evaluation datasets.
- Automated tests, lint configuration and clean-clone setup.
- README, architecture diagram, demo script, screenshots, evaluation report and limitations.

16 Access required to begin implementation

Development can start locally without external credentials. The following items may be added later through the environment configuration rather than shared in source files:

- Telegram bot token from BotFather.
- PostgreSQL connection string, if a hosted database is required.
- Optional translation, LLM or notification provider credentials.
- GitHub repository or submission instructions.
- Any organizer-specific dashboard, presentation or video requirements.

17 Reference resources

AI4Bharat: https://ai4bharat.iitm.ac.in

Bhashini: https://bhashini.gov.in

Model families to evaluate: Hugging Face IndicTrans2, MuRIL, IndicBERT and multilingual-e5. Exact model versions and licenses will be recorded in the final README.